
VIT®
BHOPAL
www.vitbhopal.ac.in

# CSE 2006 - Programming in Java

## Course Type: LP Credits: 3

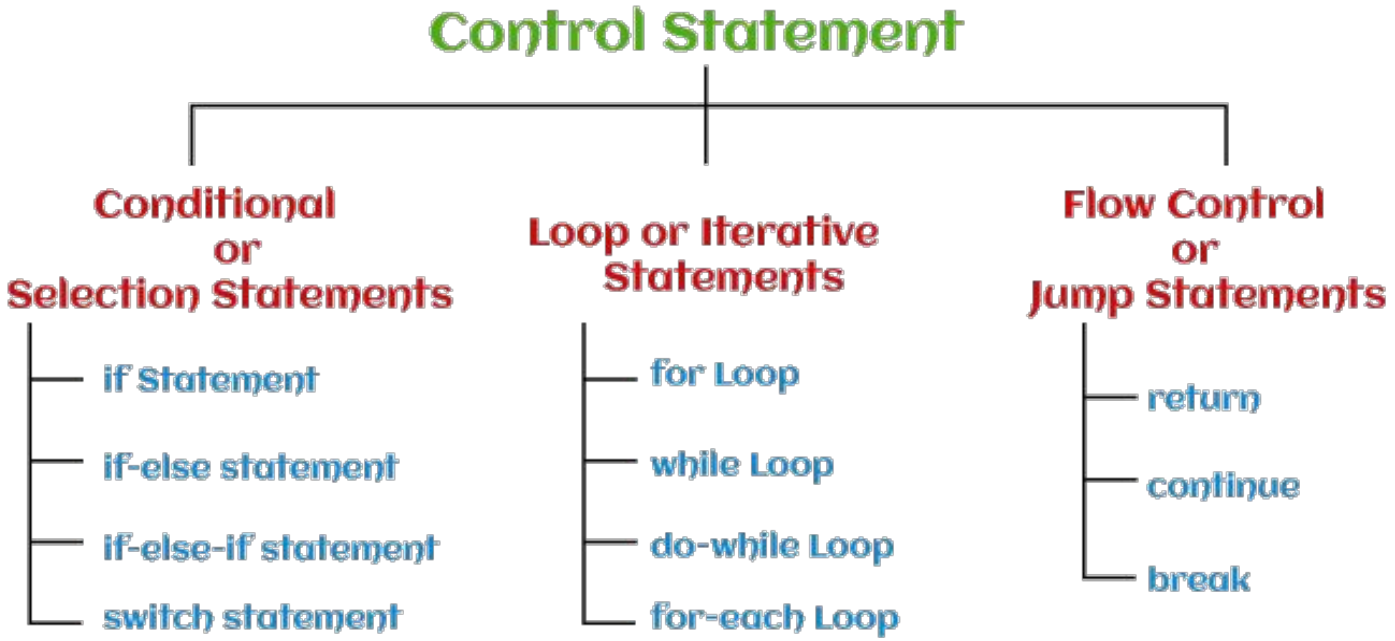
**Prepared by**
**Dr Komarasamy G**
**Senior Associate Professor**
**School of Computing Science and Engineering**
**VIT Bhopal University**

# Unit-1 Contents

- **Java Introduction**
- Java Hello World, Java JVM, JRE and JDK, Difference between C & C++, Java Variables, Java Data Types, Java Operators, Java Input and Output, Java Expressions & Blocks, Java Comment
- **Java Flow Control**
- Java if...else, Java switch Statement, Java for Loop, Java for-each Loop, Java while Loop, Java break Statement, Java continue Statement

# Java Flow Control

- **Control Statement**
- Control statements decide the flow (order or sequence of execution of statements) of a Java program. In Java, statements are parsed from top to bottom. Therefore, using the control flow statements can interrupt a particular section of a program based on a certain condition.




https://www.javatpoint.com/types-of-statements-in-java

# Java Flow Control

- here are the following types of control statements:
- **Conditional or Selection Statements**
  - if Statement
  - if-else statement
  - if-else-if statement
  - Switch statement

- **Flow Control or Jump Statements**
  - return
  - continue
  - break

**Loop or Iterative Statements**

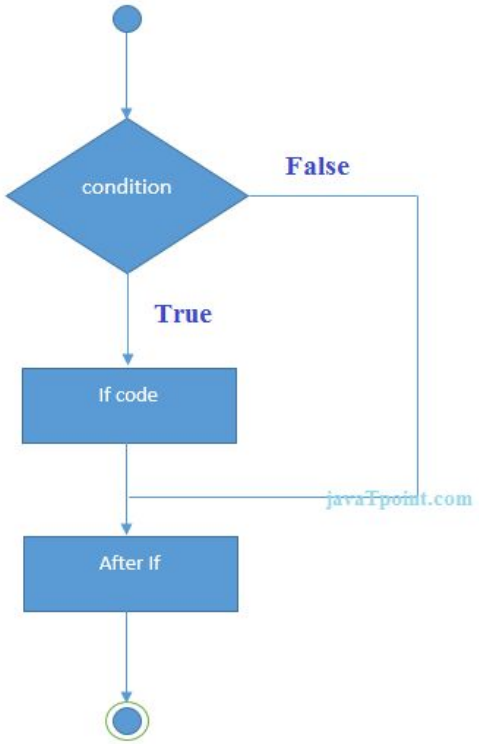
For Loop
While Loop
do-while Loop
for-each Loop

# Java Flow Control

- **Java if Statement**
- The Java if statement tests the condition. It executes the *if block* if condition is true.

**Syntax:**

```
if(condition){
//code to be executed
}
```



# Java Flow Control

- //Java Program to demonstate the use of if statement.

```
public class IfExample {
public static void main(String[] args) {
   //defining an 'age' variable
   int age=20;
   //checking the age
   if(age>18){
      System.out.print("Age is greater than 18");
   }
}
}
```

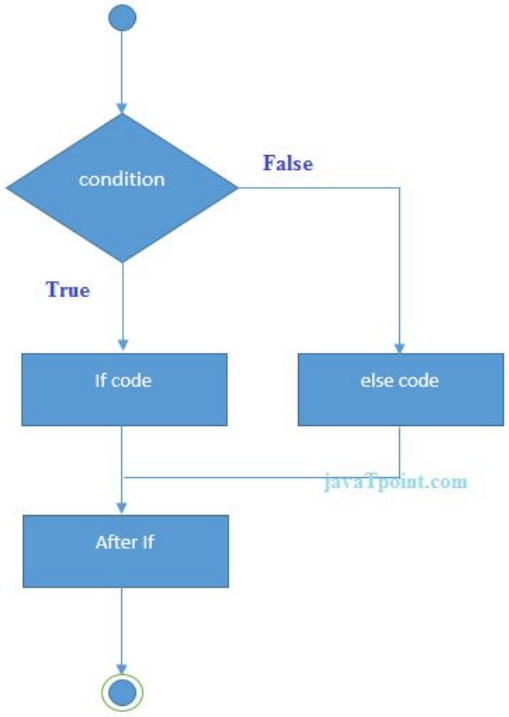
**Output**:
Age is greater than 18

# Java Flow Control

- **Java if-else Statement**
- The Java if-else statement also tests the condition. It executes the *if block* if condition is true otherwise *else block* is executed.

**Syntax:**

```
if(condition){
//code if condition is true
}else{
//code if condition is false
}
```



# Java Flow Control

- //A Java Program to demonstrate the use of if-else statement.
- //It is a program of odd and even number.

```
public class IfElseExample {
public static void main(String[] args) {
   //defining a variable
   int number=13;
   //Check if the number is divisible by 2 or not
   if(number%2==0){
      System.out.println("even number");
   }else{
      System.out.println("odd number");
   }
}
}
```

**Output**:
odd number

# Java Flow Control

- **Leap Year Example:**
- A year is leap, if it is divisible by 4 and 400. But, not by 100.

```
public class LeapYearExample {
public static void main(String[] args) {
    int year=2020;
    if(((year % 4 ==0) && (year % 100 !=0)) || (year % 400==0)){
        System.out.println("LEAP YEAR");
    }
    else{
        System.out.println("COMMON YEAR");
    }
}
}
```

**Output**:
LEAP YEAR

- **Using Ternary Operator**
- We can also use ternary operator (? :) to perform the task of if...else statement. It is a shorthand way to check the condition. If the condition is true, the result of ? is returned. But, if the condition is false, the result of : is returned.

**Example:**

```
public class IfElseTernaryExample {
public static void main(String[] args) {
   int number=13;
   //Using ternary operator
   String output=(number%2==0)?"even number":"odd number";

   System.out.println(output);
}
}
```

**Output**:
odd number

# Java Flow Control

- **Java if-else-if ladder Statement**
- The if-else-if ladder statement executes one condition from multiple statements.
- **Syntax:**

```
if(condition1){
//code to be executed if condition1 is true
}else if(condition2){
//code to be executed if condition2 is true
}
else if(condition3){
//code to be executed if condition3 is true
}
...
else{
//code to be executed if all the conditions are false
}
```

# Java Flow Control

- **Java if-else-if ladder Statement**

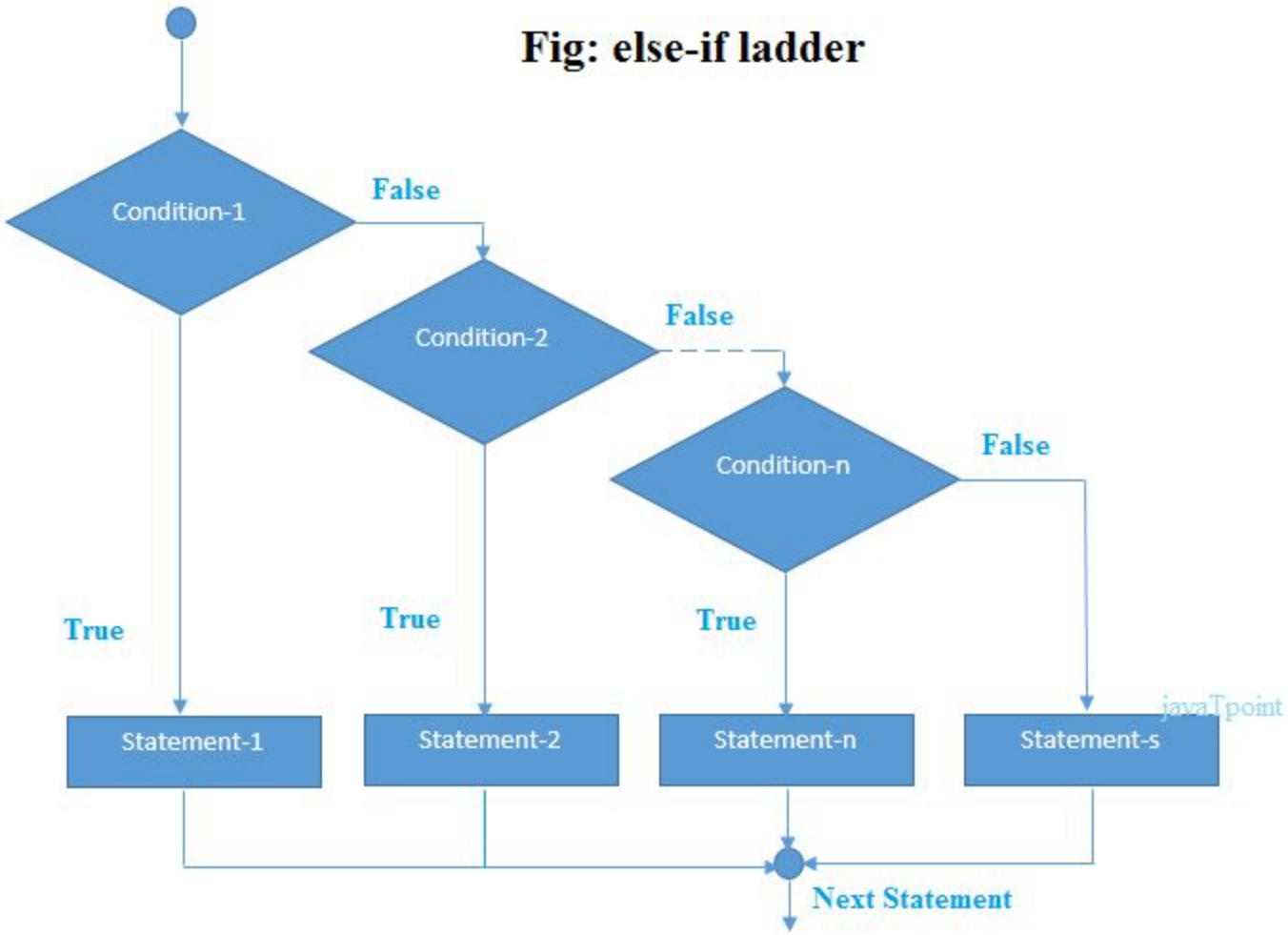



Fig: else-if ladder

# Java Flow Control

- **Example:**
- //Java Program to demonstrate the use of If else-if ladder.
- //It is a program of grading system for fail, D grade, C grade, B grade, A grad e and A+.

```
public class IfElseIfExample {
public static void main(String[] args) {
    int marks=65;

    if(marks<50){
        System.out.println("fail");
    }
    else if(marks>=50 && marks<60){
        System.out.println("D grade");
    }
    else if(marks>=60 && marks<70){
        System.out.println("C grade");
    }
```

# Java Flow Control

```
    else if(marks>=70 && marks<80){
        System.out.println("B grade");
    }
    else if(marks>=80 && marks<90){
        System.out.println("A grade");
    }else if(marks>=90 && marks<100){
        System.out.println("A+ grade");
    }else{
        System.out.println("Invalid!");
    }
}
}
```

**Output**:
C grade

# Java Flow Control

- **Program to check POSITIVE, NEGATIVE or ZERO:**

```
public class PositiveNegativeExample {
public static void main(String[] args) {
    int number=-13;
    if(number>0){
    System.out.println("POSITIVE");
    }else if(number<0){
    System.out.println("NEGATIVE");
    }else{
    System.out.println("ZERO");
   }
}
}
```

**Output**:
NEGATIVE

# Java Flow Control

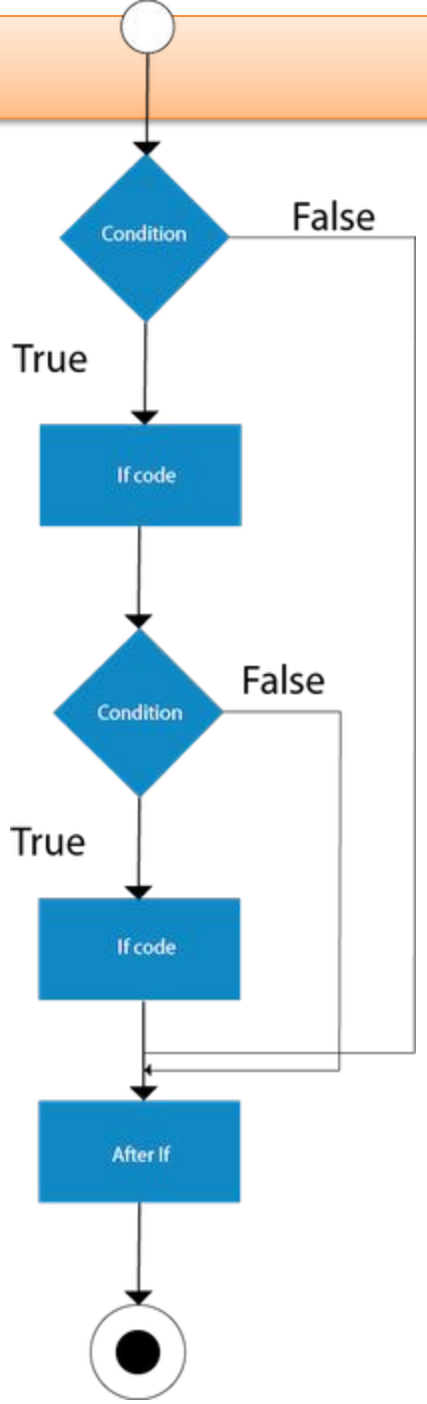
- **Java Nested if statement**
- The nested if statement represents the *if block within another if block*. Here, the inner if block condition executes only when outer if block condition is true.

**Syntax:**

```
if(condition){
    //code to be executed
        if(condition){
          //code to be executed
    }
}
```

# Java Flow Control

- **Java Nested if statement**



# Java Flow Control

- **Example:**

```
//Java Program to demonstrate the use of Nested If Statement.
public class JavaNestedIfExample {
public static void main(String[] args) {
    //Creating two variables for age and weight
    int age=20;
    int weight=80;
    //applying condition on age and weight
    if(age>=18){
        if(weight>50){
            System.out.println("You are eligible to donate blood");
        }
    }
}}
```

**Output**:
You are eligible to donate blood

# Java Flow Control

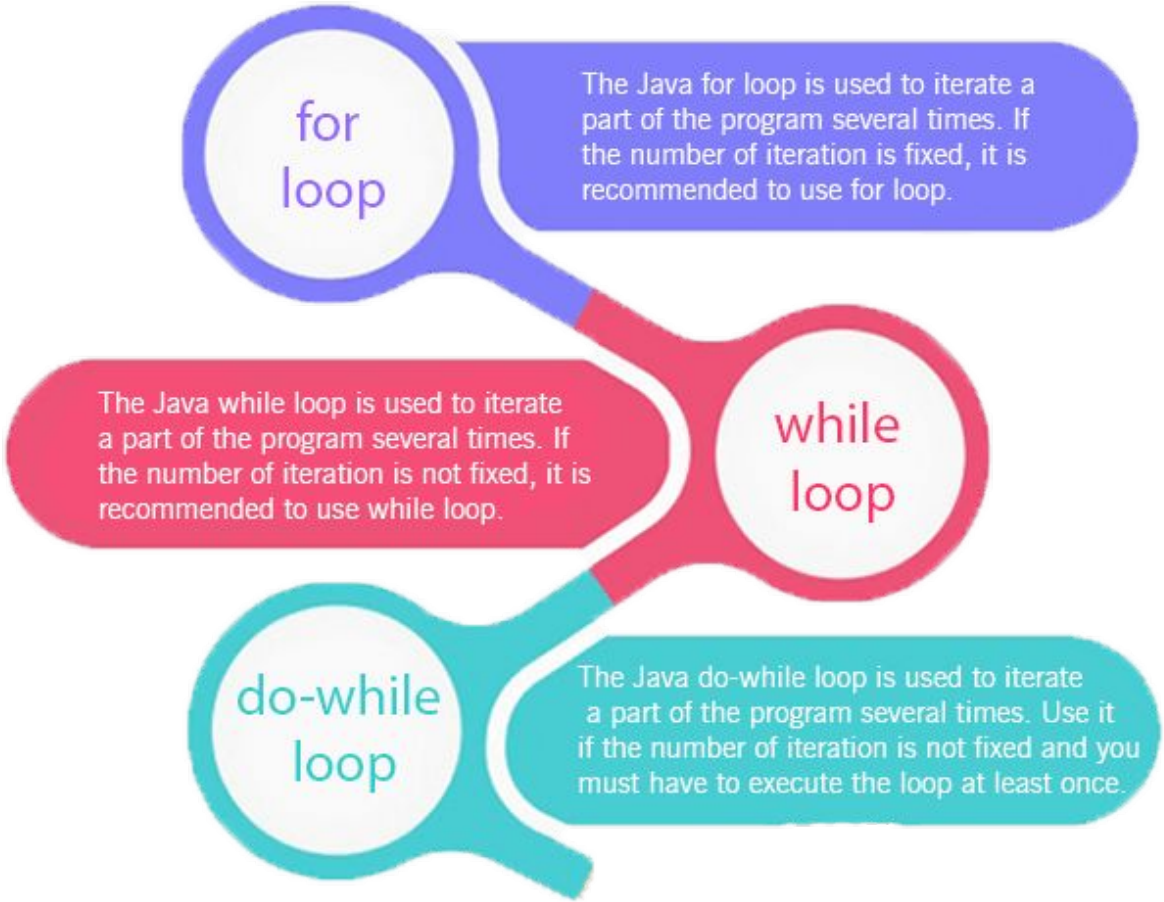
- **Example:**

```
//Java Program to demonstrate the use of Nested If Statement.
public class JavaNestedIfExample {
public static void main(String[] args) {
    //Creating two variables for age and weight
    int age=20;
    int weight=80;
    //applying condition on age and weight
    if(age>=18){
        if(weight>50){
            System.out.println("You are eligible to donate blood");
        }
    }
}}
```

**Output**:
You are eligible to donate blood

# Loops in Java

- The Java *for loop* is used to iterate a part of the program several times. If the number of iteration is **fixed**, it is recommended to use for loop. There are three types of for loops in Java.



# Java Simple for Loop

- A simple for loop is the same as C/C++. We can initialize the variable, check condition and increment/decrement value. It consists of four parts:
- **Initialization**: It is the initial condition which is executed once when the loop starts. Here, we can initialize the variable, or we can use an already initialized variable. It is an optional condition.
- **Condition**: It is the second condition which is executed each time to test the condition of the loop. It continues execution until the condition is false. It must return boolean value either true or false. It is an optional condition.
- **Increment/Decrement**: It increments or decrements the variable value. It is an optional condition.
- **Statement**: The statement of the loop is executed each time until the second condition is false.

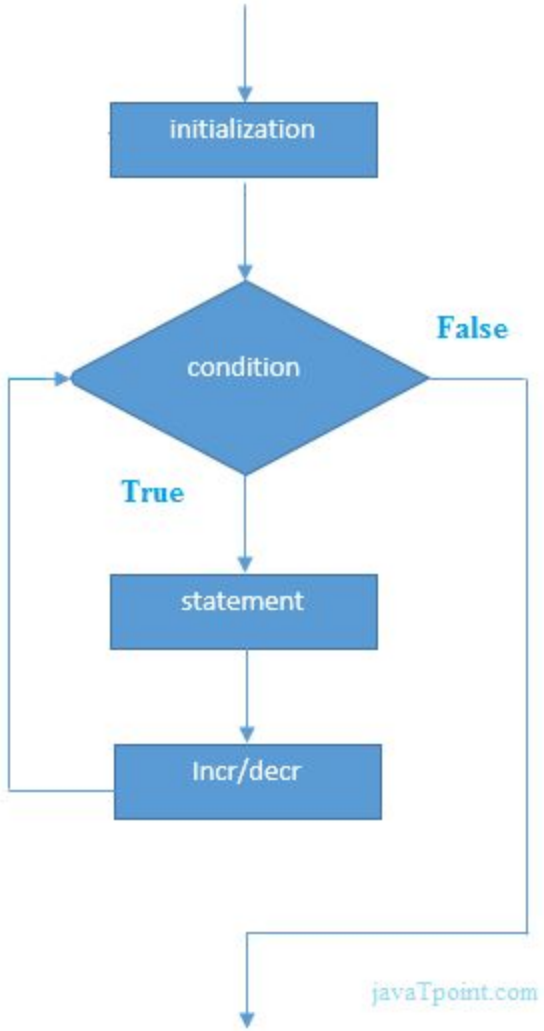
# Java Simple for Loop

- **Syntax**

**for**(initialization; co

//statement or code

}



# Java Simple for Loop

- **Example:**
- **ForExample.java**

```
//Java Program to demonstrate the example of for loop
//which prints table of 1
public class ForExample {
public static void main(String[] args) {
   //Code of Java for loop
   for(int i=1;i<=10;i++){
      System.out.println(i);
   }
}
}
```

**Output**:

1 2 3 4 5 6 7 8 9 10

# Java Nested for Loop

- If we have a for loop inside the another loop, it is known as nested for loop. The inner loop executes completely whenever outer loop executes.
- **NestedForExample.java**

```
public class NestedForExample {
public static void main(String[] args) {
//loop of i
for(int i=1;i<=3;i++){
//loop of j
for(int j=1;j<=3;j++){
      System.out.println(i+" "+j);
}//end of i
}//end of j
}
}
```

**Output**:

```
1 1
1 2
1 3
2 1
2 2
2 3
3 1
3 2
3 3
```

# Java Nested for Loop

- **Pyramid Example 1:**
- **PyramidExample.java**

```
public class PyramidExample {
public static void main(String[] args) {
for(int i=1;i<=5;i++){
for(int j=1;j<=i;j++){
        System.out.print("* ");
}
System.out.println();//new line
}
}
}
```

**Output:**

```
*
* *
* * *
* * * *
* * * * *
```

# Java for-each Loop

- The for-each loop is used to traverse array or collection in Java. It is easier to use than simple for loop because we don't need to increment value and use subscript notation.
- It works on the basis of elements and not the index. It returns element one by one in the defined variable.

- **Syntax:**

```
for(data_type variable : array_name){
//code to be executed
}
```

# Java Nested for Loop

- **Example: ForEachExample.java**

```
//Java For-each loop example which prints the
//elements of the array
public class ForEachExample {
public static void main(String[] args) {
    //Declaring an array
    int arr[]={12,23,44,56,78};
    //Printing array using for-each loop
    for(int i:arr){
        System.out.println(i);
    }
}
}
```

**Output**:

```
12
23
44
56
78
```

# Java Infinitive for Loop

- If you use two semicolons ;; in the for loop, it will be infinitive for loop.

**Syntax:**

```
for(;;){
//code to be executed
}
```

# Java Infinitive for Loop

- **Example:**
- **ForExample.java**

```
//Java program to demonstrate the use of infinite for loop
//which prints an statement
public class ForExample {
public static void main(String[] args) {
   //Using no condition in for loop
   for(;;){
      System.out.println("infinitive loop");
   }
}
}
```

**Output**:
infinitive loop
infinitive loop
infinitive loop
infinitive loop
infinitive loop ctrl+c

https://www.javatpoint.com/java-for-loop

# Java While Loop

- The Java *while loop* is used to iterate a part of the program repeatedly until the specified Boolean condition is true. As soon as the Boolean condition becomes false, the loop automatically stops.
- The while loop is considered as a repeating if statement. If the number of iteration is not fixed, it is recommended to use the while loop.

**Syntax:**

**while** (condition){

//code to be executed

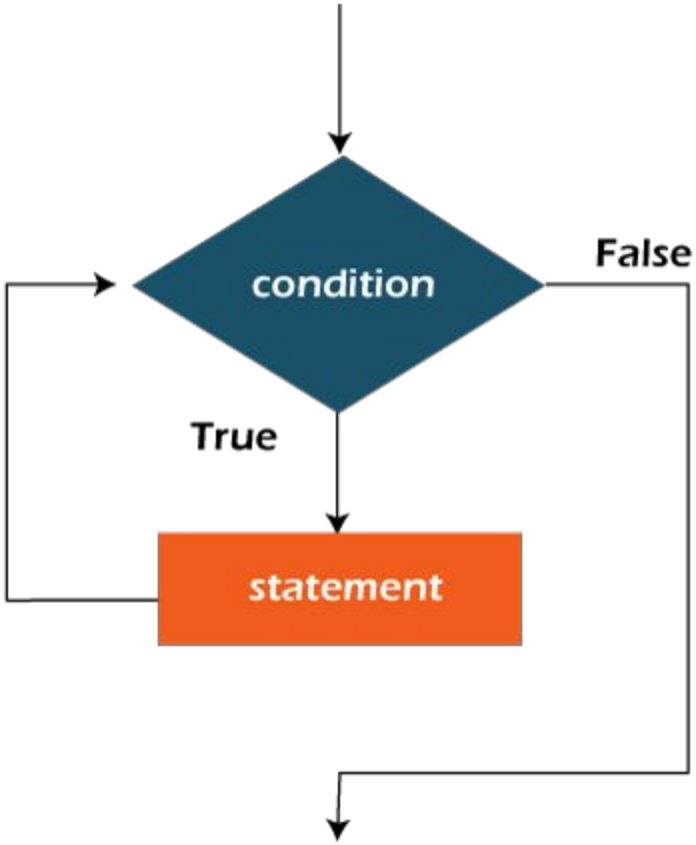
I ncrement / decrement statement

}

https://www.javatpoint.com/java-while-loop

# Java While Loop

- **Flowchart**



# Java While Loop

- In the below example, we print integer values from 1 to 10. Unlike the for loop, we separately need to initialize and increment the variable used in the condition (here, i). Otherwise, the loop will execute infinitely.

**WhileExample.java**

```
public class WhileExample {
public static void main(String[] args) {
   int i=1;
   while(i<=10){
      System.out.println(i);
   i++;
   }
}
}
```

**Output**:

1 2 3 4 5 6 7 8 9 10

https://www.javatpoint.com/java-while-loop

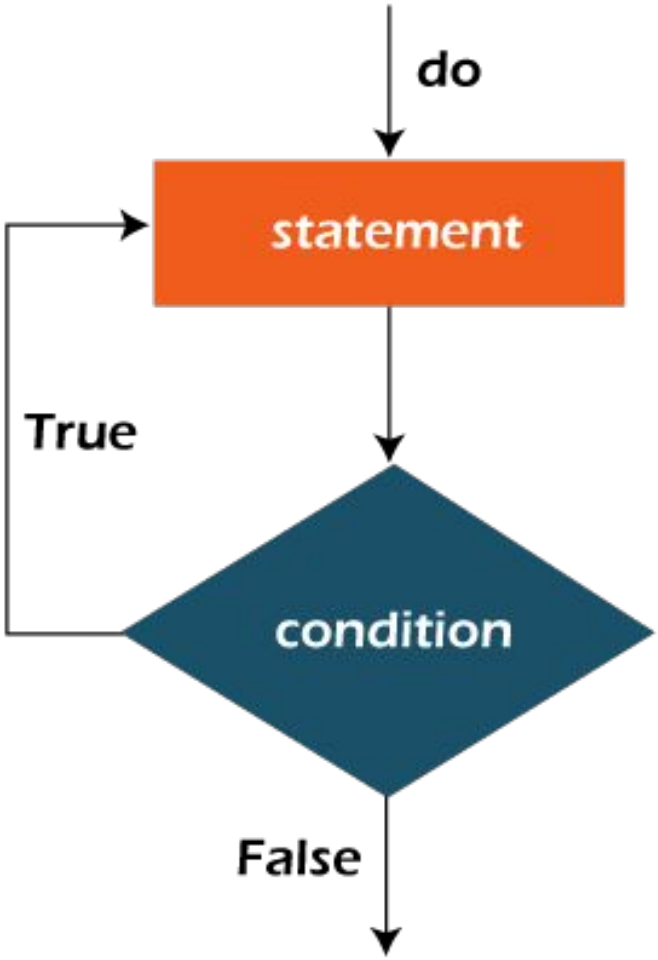
# Java do-while Loop

- The Java *do-while loop* is used to iterate a part of the program repeatedly, until the specified condition is true.
- If the number of iteration is not fixed and you must have to execute the loop at least once, it is recommended to use a do-while loop. Java do-while loop is called an **exit control loop**.
- Therefore, unlike while loop and for loop, the do-while check the condition at the end of loop body. The Java *do-while loop* is executed at least once because condition is checked after loop body.
- **Syntax:**

```
do{
//code to be executed / loop body
//update statement
}while (condition);
```

# Java do-while Loop

- **Flowchart of do-while loop:**



# Java do-while Loop

- In the below example, we print integer values from 1 to 10. Unlike the for loop, we separately need to initialize and increment the variable used in the condition (here, i). Otherwise, the loop will execute infinitely.
- **DoWhileExample.java**

```
public class DoWhileExample {
public static void main(String[] args) {
    int i=1;
    do{
        System.out.println(i);
    i++;
    }while(i<=10);
}
}
```

**Output**:
1 2 3 4 5 6 7 8 9 10

https://www.javatpoint.com/java-do-while-loop

# Java Break Statement

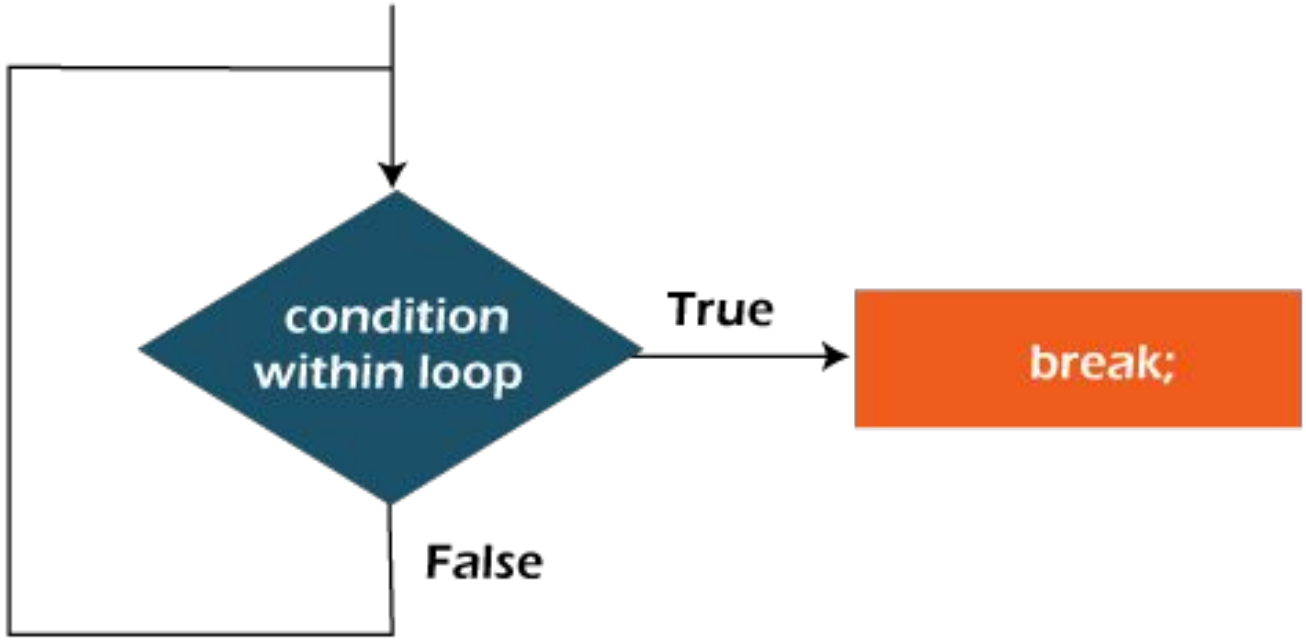
- When a break statement is encountered inside a loop, the loop is immediately terminated and the program control resumes at the next statement following the loop.
- The Java *break* statement is used to break loop or switch statement. It breaks the current flow of the program at specified condition. In case of inner loop, it breaks only inner loop.
- We can use Java break statement in all types of loops such as for loop, while loop and do-while loop.
- **Syntax:**

jump-statement;

**break**;

# Java Break Statement

- **Flowchart of Break Statement**




Flowchart of break statement

# Java Break Statement with Loop

- **Example: BreakExample.java**
- //Java Program to demonstrate the use of break statement

```
public class BreakExample {
public static void main(String[] args) {
   //using for loop
   for(int i=1;i<=10;i++){
      if(i==5){
         //breaking the loop
         break;
      }
      System.out.println(i);
   }
}
}
```

**Output**:

1
2
3
4

https://www.javatpoint.com/java-break

# Java Continue Statement

- The continue statement is used in loop control structure when you need to jump to the next iteration of the loop immediately. It can be used with for loop or while loop.
- The Java *continue statement* is used to continue the loop. It continues the current flow of the program and skips the remaining code at the specified condition. In case of an inner loop, it continues the inner loop only.
- We can use Java continue statement in all types of loops such as for loop, while loop and do-while loop.

- **Syntax:**

jump-statement;

**continue**;

# Java Continue Statement

- **ContinueExample.java**
- //Java Program to demonstrate the use of continue statement

```
public class ContinueExample {
public static void main(String[] args) {
   //for loop
   for(int i=1;i<=10;i++){
      if(i==5){
         //using continue statement
         continue;//it will skip the rest statement
      }
      System.out.println(i);
   }
}
}
```

**Output**:

1 2 3 4 5 6 7 8 9 10

**https://www.javatpoint.com/java-continue**

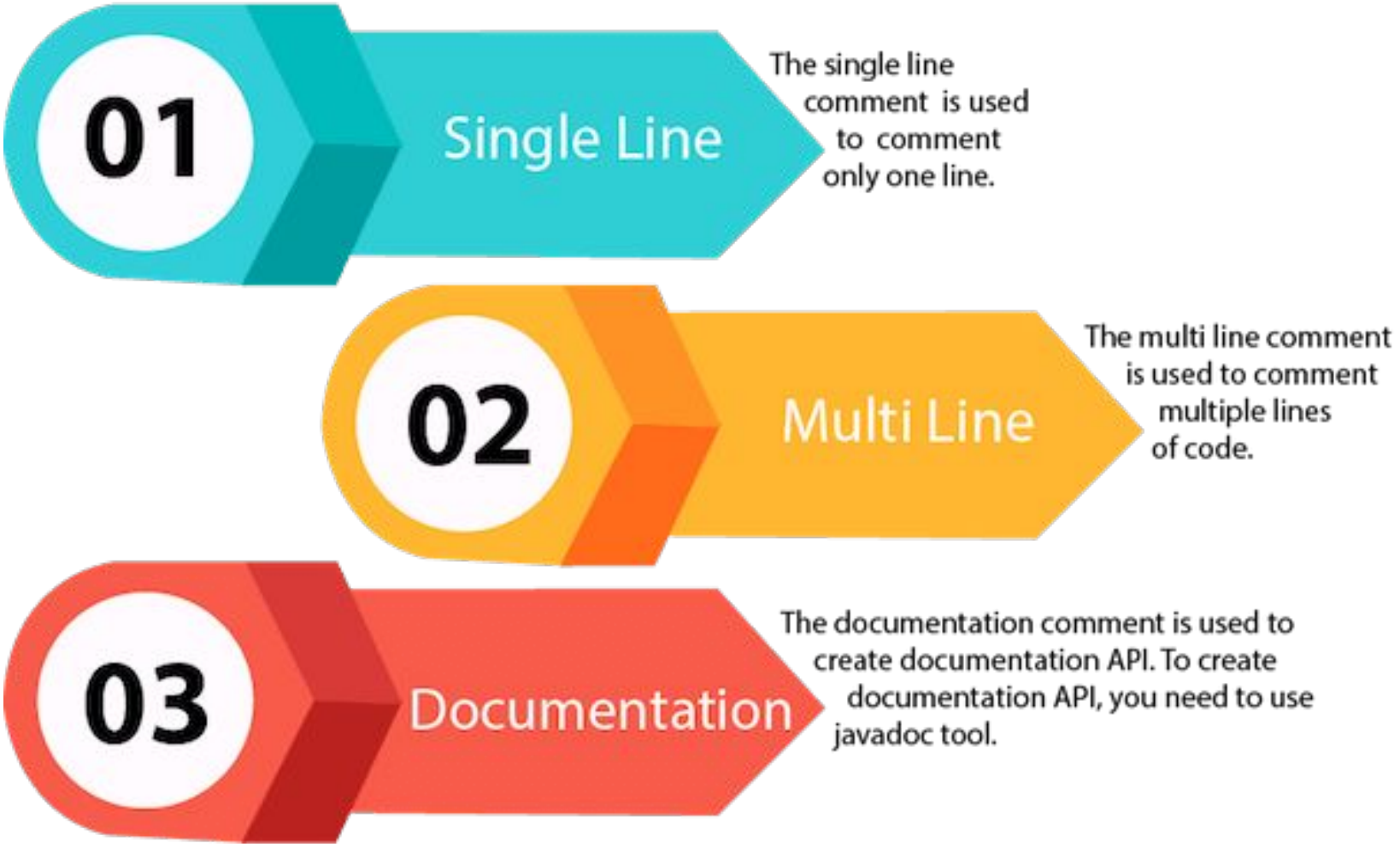
# Java Comments

- The Java comments are the statements in a program that are not executed by the compiler and interpreter.
- Why do we use comments in a code?
- Comments are used to make the program more readable by adding the details of the code.
- It makes easy to maintain the code and to find the errors easily.
- The comments can be used to provide information or explanation about the variable, method, class, or any statement.
- It can also be used to prevent the execution of program code while testing the alternative code.
- **Types of Java Comments**

**There are three types of comments in Java.**

- Single Line Comment
- Multi Line Comment
- Documentation Comment

# Types of Java Comments



# Java more example programs

- https://www.javatpoint.com/java-programs

# Java Links

**How to Run Java Program in Command Prompt (JDK Installation)**
https://youtu.be/F3Mh6NS69Z0

**Java Programming reference Links**
https://www.javatpoint.com/java-tutorial
https://www.w3schools.com/java/default.asp
https://www.tutorialspoint.com/java/index.htm
https://simplesnippets.tech/courses/core-java-programming-tutorials-for-beginners/

**Java Full Course In 11 Hours | Java Programming |Simplilearn**
https://youtu.be/CFD9EFcNZTQ

**Simply learn for java**
https://www.youtube.com/hashtag/javatutorialforbeginners